



HCMC University of Technology, VNU

# Software Architecture



**Lecturer: Thai-Minh Truong, PhD.**

**Email: [thaiminh@hcmut.edu.vn](mailto:thaiminh@hcmut.edu.vn)**



## **CHAPTER 3: ARCHITECTURAL THINKING & MODULARITY**

# Chapter 3: Architectural Thinking & Modularity

---



3.1. Architecture vs Design

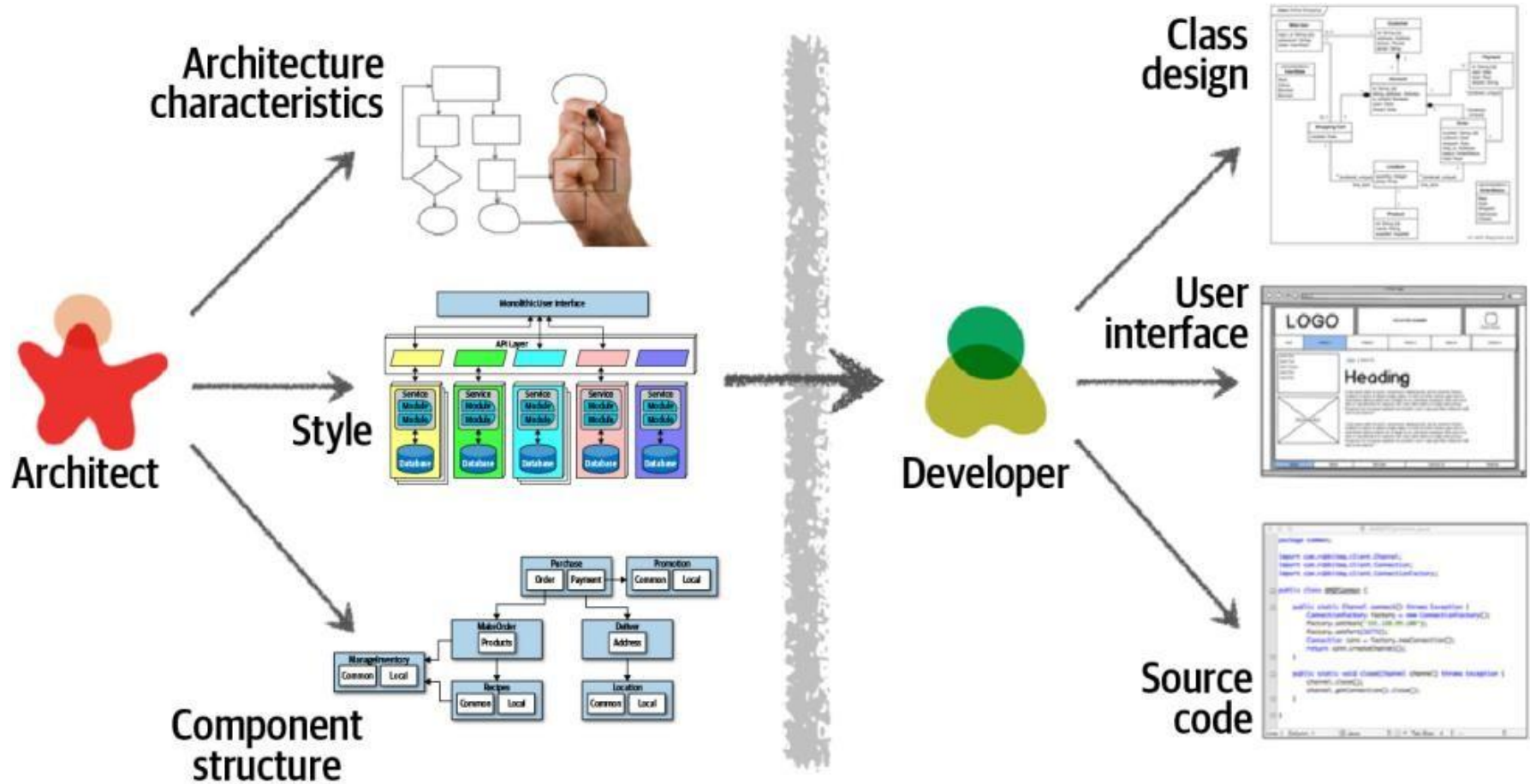
3.2. Analyzing Trade-Offs

3.3. Balancing Architecture and Hands-On  
Coding

3.4. Modularity

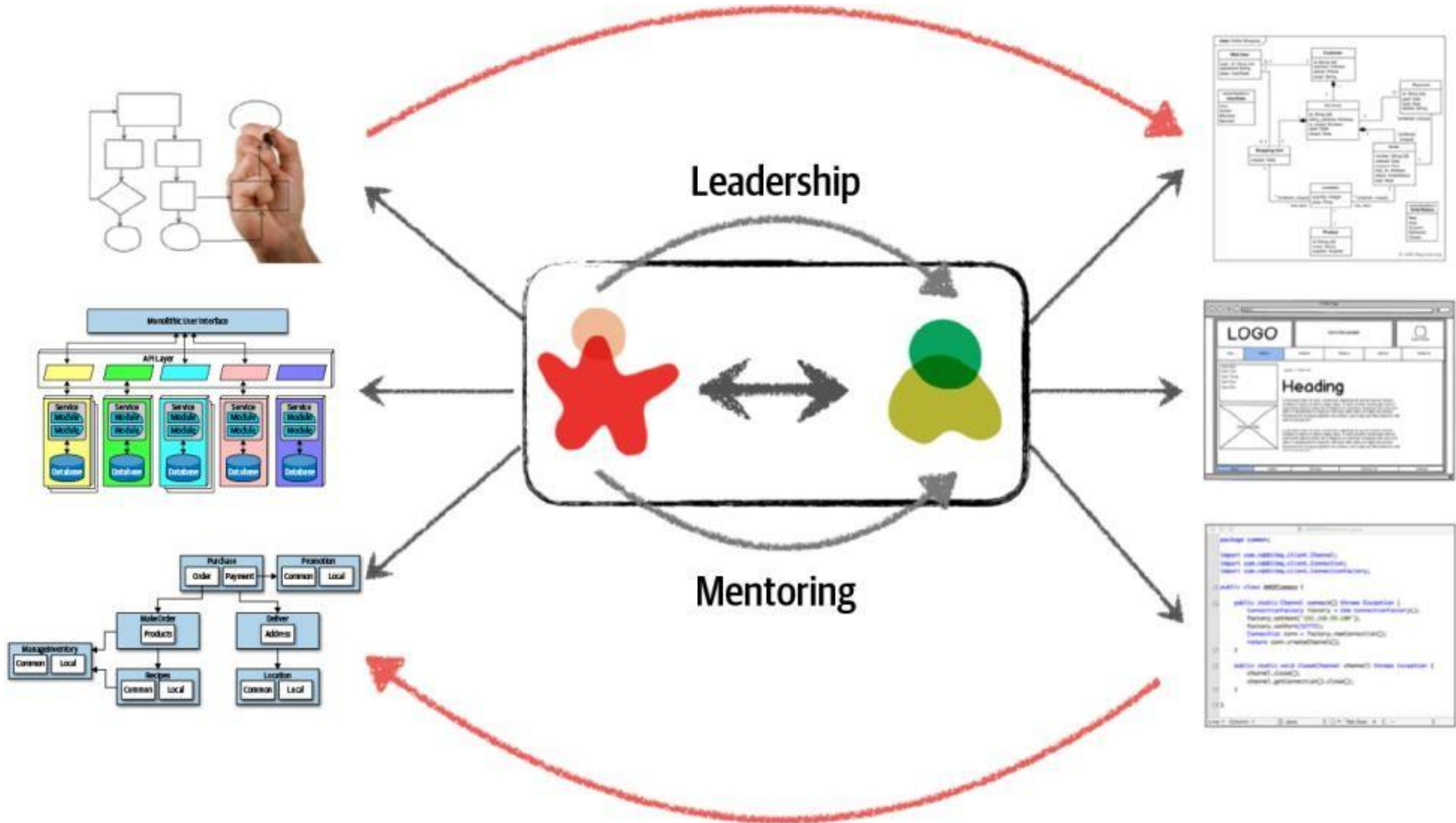
# 3.1. Architecture vs Design

## Traditional View : Architecture vs Design



# 3.1. Architecture vs Design

Making architecture work through *collaboration*

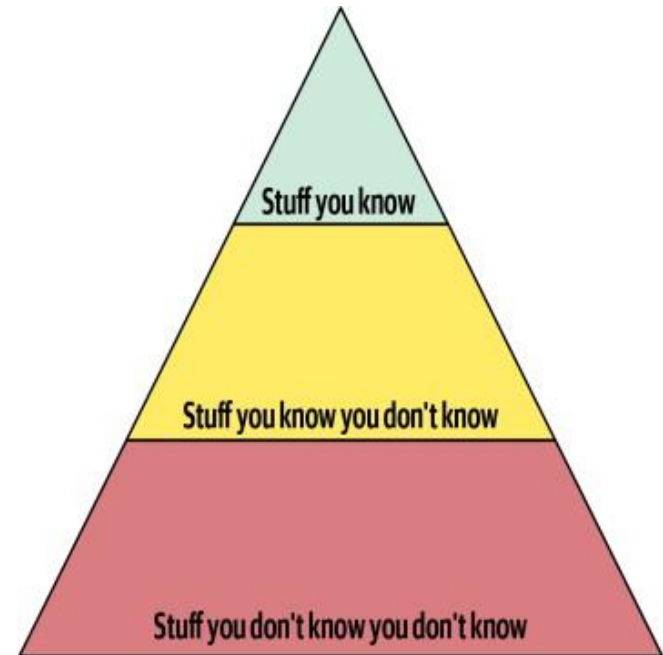


# Levels of knowledge



## *Knowledge triangle*

- ▶ **Stuff you know** - includes the technologies, frameworks, languages, and tools a technologist uses on a daily basis to perform their job
- ▶ **Stuff you know you don't know** - includes those things a technologist knows a little about or has heard of but has little or no expertise in
- ▶ **Stuff you don't know you don't know** - includes the entire host of technologies, tools, frameworks, and languages that would be the perfect solution to a problem a technologist is trying to solve, but the technologist doesn't even know those things exist



# Technical Breadth

*Technical **Depth** and Technical **Breadth***

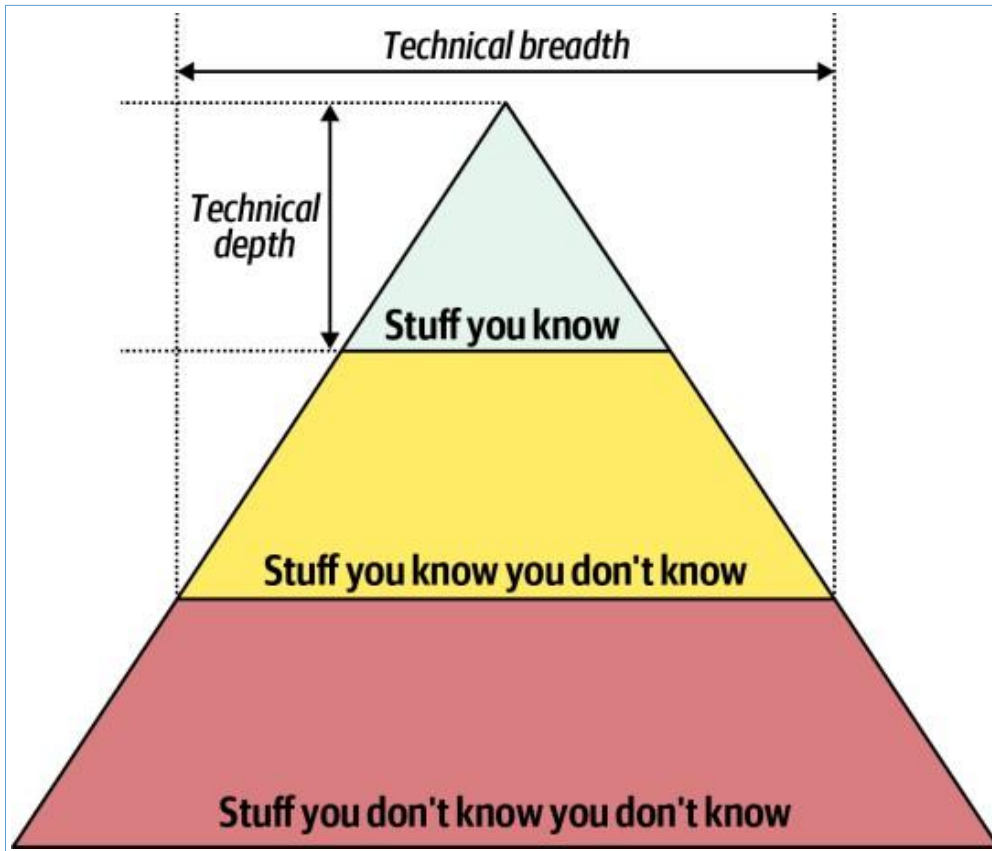
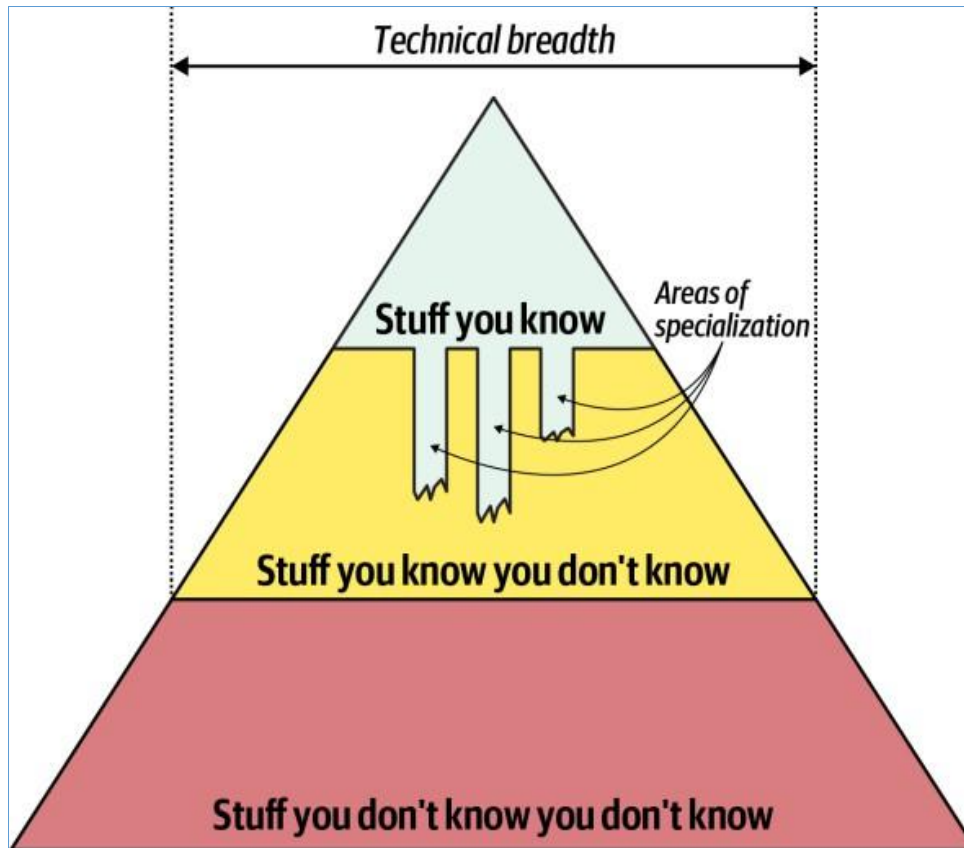


Figure 2.5. What someone knows is technical depth, and how much someone knows is technical breadth

# Technical Breadth



*Developers* will focus on Technical **Depth** and *Architect* on Technical **Breadth**



Why is it more important for an architect to focus on technical breadth rather than technical depth?

Figure 2-6. Enhanced breadth and shrinking depth for the architect role

## 3.2. Analyzing Trade-Offs



- ▶ *Everything* in architecture is a **trade-off**, which is why the famous answer to every architecture question in the universe is “**it depends.**”
  - It depends on:
    - the deployment environment,
    - business drivers,
    - company culture, budgets,
    - timeframes,
    - developer skill set,
    - and dozens of other factors.
  - Everyone’s environment, situation, and problem is different, hence why architecture is so hard.

There are no right or wrong answers in architecture—only trade-offs. (Neal )

## 3.2. Analyzing Trade-Offs



### Example

- Consider an item **auction system**, where someone places a bid for an item up for auction.
- The **Bid Producer** service generates a bid from the bidder and then sends that bid amount to **the Bid Capture, Bid Tracking, and Bid Analytics** services.
- Using **queues** in a point-to-point messaging fashion or a **topic** in a publish-and-subscribe messaging fashion?

**Architectural thinking:**  
*analyze the trade-offs and select the best one given the specific situation.*

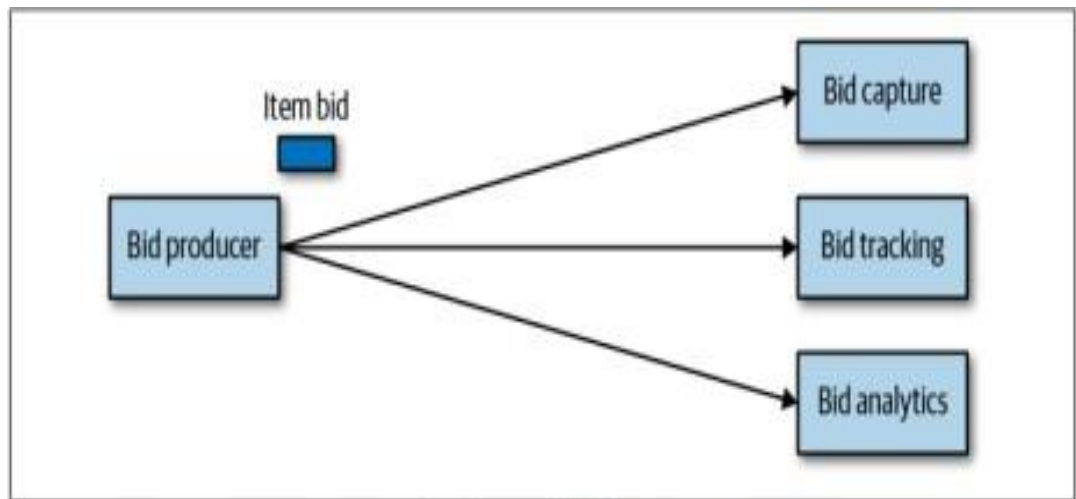


Figure 2-7. Auction system example of a trade-off—queues or topics?

## 3.2. Analyzing Trade-Offs



### *Example (Topic)*

- The Bid Producer service **only requires** a **single connection** to a topic.
- The Bid Producer **doesn't know how** the bidding information will **be used** or **by which** services.
- The topic solution only supports **homogeneous contracts**.
- All services receiving the bidding data **must accept** the **same contract** and set of bidding data.
- Not support monitoring of the number of messages in the topic.

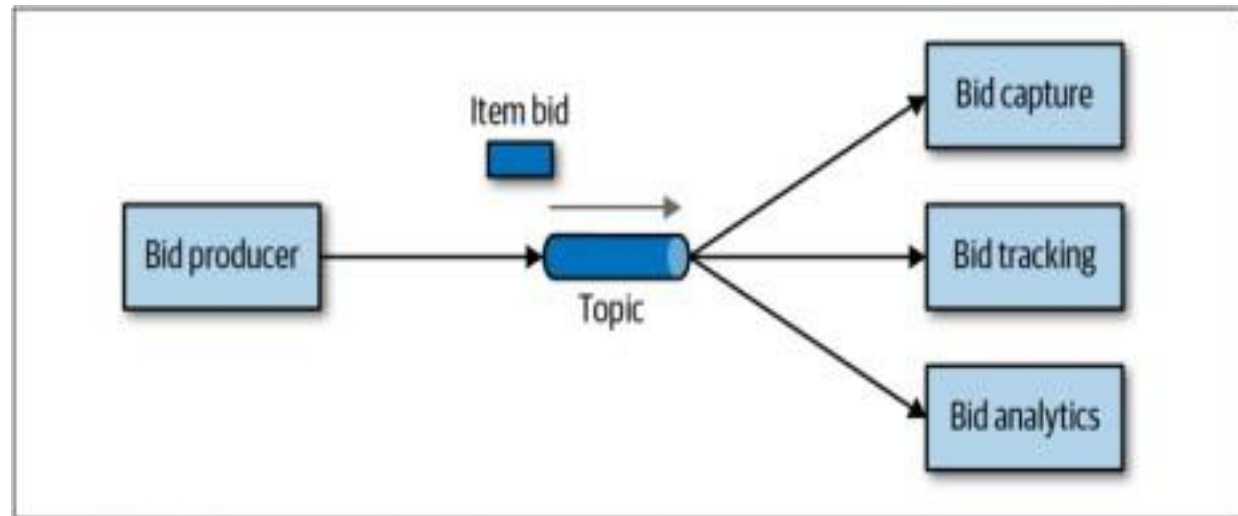


Figure 2-8. Use of a topic for communication between services

## 3.2. Analyzing Trade-Offs



### *Example (Queues)*

- The Bid Producer needs to **connect** to **three different queues**.
- The Bid Producer **knows exactly how** the bidding information is **used** (and **by whom**).
- Each consumer can have **its own contract** specific to the data it needs.
- Each queue can be monitored individually.

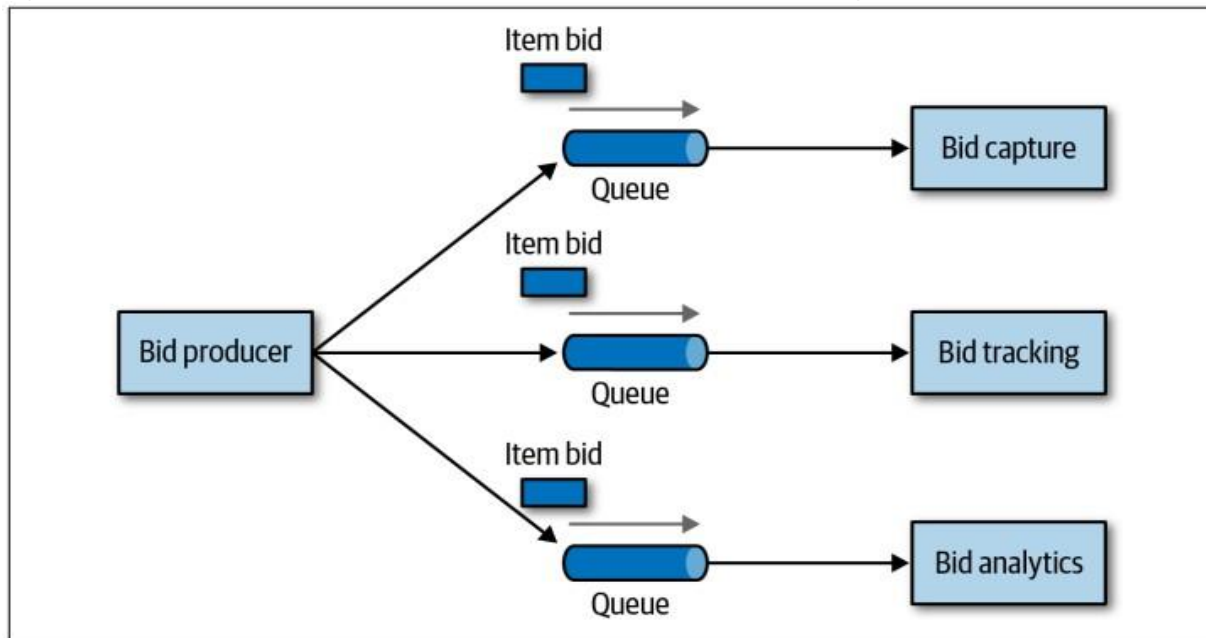


Figure 2-9. Use of queues for communication between services

## 3.2. Analyzing Trade-Offs



### Example

Analyze advantages & disadvantages

- **S1:** What if a **new service** called Bid History were to be **added** to the system?
- **S2:** How about the **data access** and **data security**?
- **S3:** What if the **new** Bid History **service requires** the current asking **price** along with the bid, but **no other service needs** that information?
- **S4:** How about the **monitoring** and **programmatic scalability**?

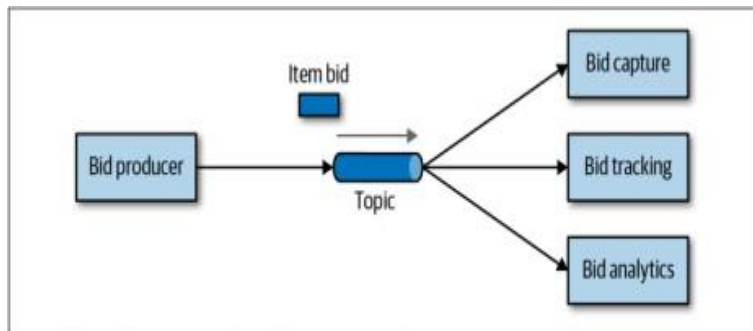


Figure 2-8. Use of a topic for communication between services

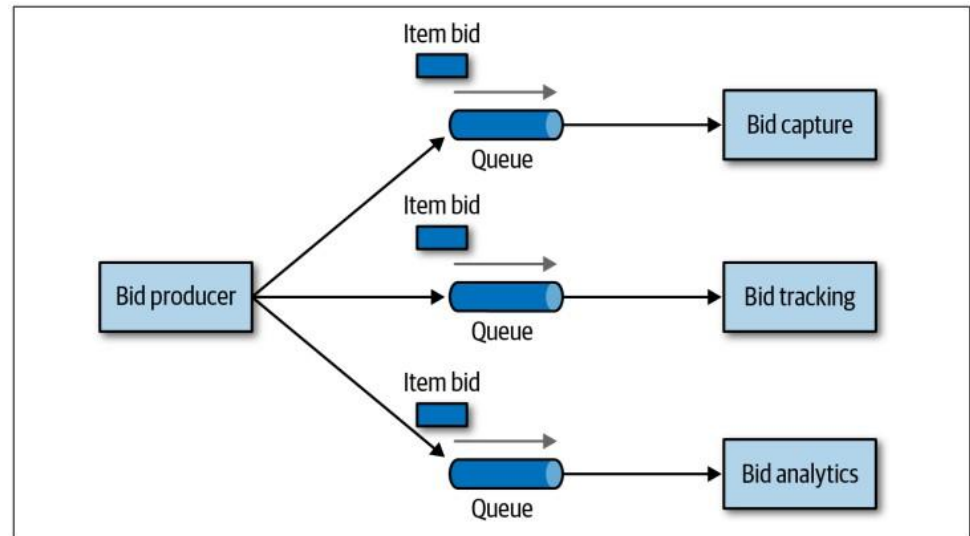


Figure 2-9. Use of queues for communication between services

## 3.2. Analyzing Trade-Offs (examples)



- Performance vs. Maintainability
  - Optimization requires the use of complex techniques → code becomes difficult to understand, hard to maintain, etc.
- Scalability vs. Simplicity
  - Good scalability often leads to complex systems such as applying load balancing, microservices, etc. → not simple.
- Security vs. User Experience
  - Enhancing security requires multi-layer authentication → reduces user experience
- Development Cost vs. Quality
  - Reducing costs → using open source → impacts quality

# Understanding Business Drivers



*What are business drivers?*

- ▶ **Business drivers** are the major inputs and actions that **drive** a company's **operational** and **financial success**. For example, this may include salespeople, the number of stores, online traffic, the number and price of products sold and production units.
  - Businesses may use these **key drivers** to determine how to improve their company and often aim to maximize any within their control.
- ▶ Thinking like an architect is **understanding** the **business drivers** required for the system's success and **translating** those **requirements** into **architectural characteristics** (such as scalability, performance, and availability).
- ▶ This is a **challenging task** that requires the **architect** to **have** some level of **business domain knowledge** and healthy, collaborative **relationships** with **key business stakeholders**.

# 3.3. Balancing Architecture and Coding



## *Balancing Architecture and Hands-On Coding*

- ▶ Striving for a **balance** between **hands-on coding** and being a **software architect** is **avoiding** the **bottleneck** trap.
  - The **bottleneck trap occurs** when the architect has taken ownership of code within the critical path of a project and becomes a bottleneck to the team.
  - To **avoid the bottleneck trap** as an effective software architect is to delegate the critical path and framework code to others on the development team and then focus on coding a piece of business functionality (a service or a screen) one to three iterations down the road.
    - gaining **hands-on experience** writing production **code** while no longer becoming a bottleneck on the team
    - the critical path and framework code is distributed to the **development team** (where it belongs), giving them **ownership** and a **better understanding** of the **harder parts** of the system.
    - writing the same business-related source code as the development team and is therefore better able to **identify** with the development team in terms of the **pain** they might be going through with **processes, procedures**, and the **development environment**.

# 3.3. Balancing Architecture and Coding



## *Balancing Architecture and Hands-On Coding*

- ▶ **Four basic ways** an architect can still remain hands-on at work without having to “practice coding from home”
  - To **do frequent proof-of-concepts** (POCs). Not only requires to write source code, but it also helps validate an architecture decision.
  - **Tackle** some of the **technical debt stories** or **architecture stories**, freeing the development team up to work on the critical functional user stories.
  - **Working on bug fixes** within an iteration is another way of maintaining hands-on coding while helping the development team as well.
  - **Leveraging automation** by **creating simple** command-line **tools** and analyzers to help the development team.

## 3.4. Modularity

### Definition

"95% of the words [about software architecture] are spent extolling the benefits of "modularity" and that little, if anything, is said about how to achieve it."

—Glenford J. Myers (1978)

- ▶ The dictionary defines module as *"each of a set of standardized parts or independent units that can be used to construct a more complex structure."*
- ▶ We use **modularity** to describe a **logical grouping of related code**, which could be a group of classes in an object-oriented language or functions in a structured or functional language.
  - Most languages provide mechanisms for modularity (package in Java, namespace in .NET, and so on).
  - Developers typically use modules as a way to group related code together.
    - For example, the `com.mycompany.customer` package in Java should contain things related to customers.
- ▶ Architects must be aware of how developers package things because it has important implications in architecture.
  - Researchers created a variety of language-agnostic metrics to help architects understand modularity: **cohesion**, **coupling**, and **connascence**

## 3.4. Measuring Modularity



### *Cohesion*

- ▶ Cohesion refers to what extent the parts of a module should be contained within the same module.
  - In other words, it is a measure of how related the parts are to one another.
  - Ideally, a cohesive module is one where all the parts should be packaged together, because breaking them into smaller pieces would require coupling the parts together via calls between modules to achieve useful results.
- ▶ Cohesion metrics measure how well the methods of a class are related to each other. A cohesive class performs one function while a non-cohesive class performs two or more unrelated functions. A non-cohesive class may need to be restructured into two or more smaller classes.

## 3.4. Measuring Modularity



- 📖 **High cohesion** is **desirable** since it promotes encapsulation. As a drawback, a highly cohesive class has high coupling between the methods of the class, which in turn indicates high testing effort for that class.
- 📖 **Low cohesion** indicates **inappropriate design** and **high complexity**. It has also been found to indicate a high likelihood of errors. The class should probably be split into two or more smaller classes.
- 📖 *Attempting to divide a cohesive module would only result in increased coupling and decreased readability.*

Larry Constantine

# 3.4. Measuring Modularity



## *Cohesion listing*

### ▶ Functional cohesion

- Every part of the module is related to the other, and the module contains everything essential to function.

### ▶ Sequential cohesion

- Two elements interact, where one outputs data that becomes the input for the other.

### ▶ Communicational cohesion

- Two elements form a communication chain, where each operates on information and/or contributes to some output. For example, add a record to the database and generate an email based on that information.

### ▶ Procedural cohesion

- Two elements must execute code in a particular order.

## 3.4. Measuring Modularity



### *Cohesion listing (cont.)*

#### ► Temporal cohesion

- Elements are related based on timing dependencies. For example, many systems have a list of seemingly unrelated things that must be initialized at system startup; these different tasks are temporally cohesive.

#### ► Logical cohesion

- The data within modules is related logically but not functionally. For example, consider a module that converts information from text, serialized objects, or streams. Operations are related, but the functions are quite different. A common example of this type of cohesion exists in virtually every Java project in the form of the StringUtils package: a group of static methods that operate on String but are otherwise unrelated.

#### ► Coincidental cohesion

- Elements in a module are not related other than being in the same source file; this represents the most negative form of cohesion.

# 3.4. Measuring Modularity



## *The Lack of Cohesion in Methods (LCOM)*

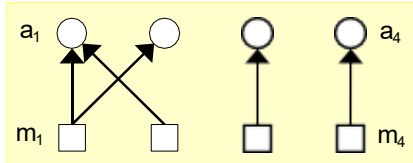
| Class Cohesion Metric                                              | Definition/Formula                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lack of Cohesion of Methods (LCOM1)<br>(Chidamber & Kemerer, 1991) | LCOM1 = Number of pairs of methods that do not share attributes                                                                                                                                                                                                                                                                                              |
| LCOM2<br>(Chidamber & Kemerer, 1991)                               | <p>P = Number of pairs of methods that do not share attributes<br/> Q = Number of pairs of methods that share attributes</p> $LCOM2 = \begin{cases} P - Q, & \text{if } P \geq Q \\ 0, & \text{otherwise} \end{cases}$                                                                                                                                       |
| LCOM3<br>(Li & Henry, 1993)<br>(Henderson-Sellers, 1996)           | $LCOM3 = \frac{m - \frac{\sum mA}{a}}{m - 1}$ <p>m: the number of methods in the class,<br/> a: the number of variables (attributes) in the class,<br/> <math>\sum mA</math>: the sum of the number of methods that access each variable, essentially the total number of method-variable access pairs.</p>                                                  |
| LCOM4<br>(Hitz & Montazeri, 1995)                                  | <p>measures the number of "connected components" in a class. A connected component is a set of related methods (and class-level variables).</p> <p>Which methods are related? Methods A and B are related if:</p> <ul style="list-style-type: none"> <li>• they both access the same class-level variable, or</li> <li>• A calls B or vice versa.</li> </ul> |

# 3.4. Measuring Modularity

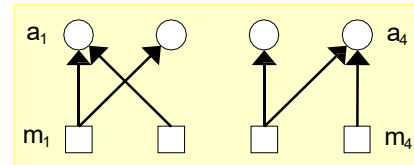
## The Lack of Cohesion in Methods (LCOM) - examples



class C1

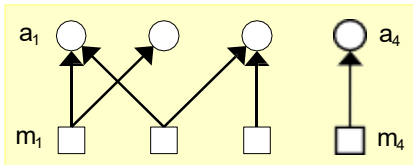


class C2

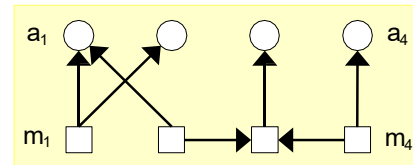


$$\# \text{ Method Pairs} = NP = \binom{M}{2} = \frac{M!}{2! \cdot (M-2)!}$$

class C3



class C4



$$NP(C_i) = \binom{4}{2} = 6$$

**LCOM1:**

$$\begin{aligned} LCOM1(C1) &= P - Q = 6 - 1 = 5 \\ LCOM1(C2) &= 6 - 2 = 4 \\ LCOM1(C3) &= 6 - 2 = 4 \\ LCOM1(C4) &= 6 - 1 = 5 \end{aligned}$$

**LCOM2:**

$$\begin{aligned} LCOM2(C1) &= P - Q = 5 - 1 = 4 \\ LCOM2(C2) &= 4 - 2 = 2 \\ LCOM2(C3) &= 4 - 2 = 2 \\ LCOM2(C4) &= 5 - 1 = 4 \end{aligned}$$

**LCOM3:**

$$\begin{aligned} LCOM3(C1) &= (4-5/4)/(4-1) = 11/12 \\ LCOM3(C2) &= (4-6/4)/(4-1) = 5/6 \\ LCOM3(C3) &= (4-6/4)/(4-1) = 5/6 \\ LCOM3(C4) &= (4-5/4)/(4-1) = 11/12 \end{aligned}$$

**LCOM4:**

$$\begin{aligned} LCOM4(C1) &= 3 \\ LCOM4(C2) &= 2 \\ LCOM4(C3) &= 2 \\ LCOM4(C4) &= 1 \end{aligned}$$

## 3.4. Measuring Modularity



### *LCOM3*

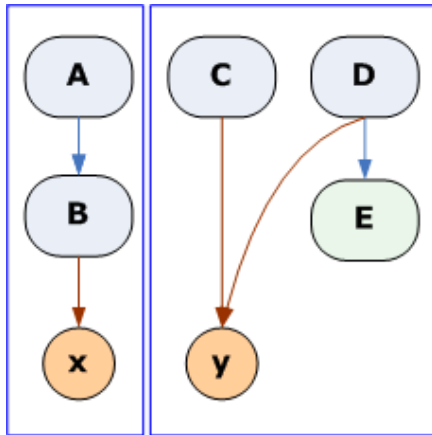
- LCOM3 varies **between 0 and 2**. Values 1..2 are considered alarming.
- In a normal class whose methods access the class's own variables, LCOM3 varies between 0 (high cohesion) and 1 (no cohesion).
- When LCOM3 = **0**, each method accesses all variables. This indicates the **highest possible cohesion**.
- LCOM3 = **1** indicates **extreme lack of cohesion**. In this case, the class should be split.
- When there are **variables** that are **not accessed by any of the class's methods**,  **$1 < \text{LCOM3} \leq 2$** . This happens if the variables are dead or they are only accessed outside the class. Both cases represent a design flaw. The class is a candidate for rewriting as a module. Alternatively, the class variables should be encapsulated with accessor methods or properties. There may also be some dead variables to remove.
- If there are **no more than one method** in a class, LCOM3 is **undefined**. If there are **no variables** in a class, LCOM3 is **undefined**. An undefined LCOM3 is **displayed as zero**.

# 3.4. Measuring Modularity

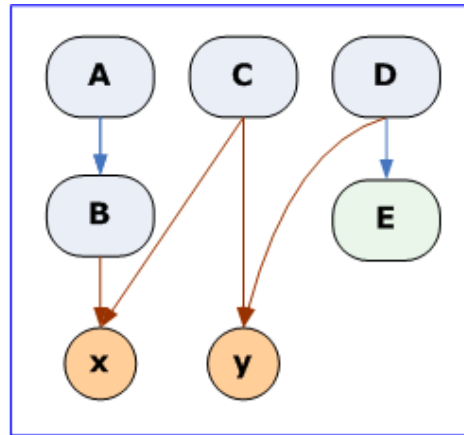


## LCOM4

- After determining the related methods, we draw a graph linking the related methods to each other. LCOM4 equals the number of connected groups of methods.
- LCOM=1 indicates a cohesive class, which is the "good" class.
- LCOM $\geq$ 2 indicates a problem. The class should be split into so many smaller classes.
- LCOM=0 happens when there are no methods in a class. This is also a "bad" class.



LCOM4 = 2



LCOM4 = 1

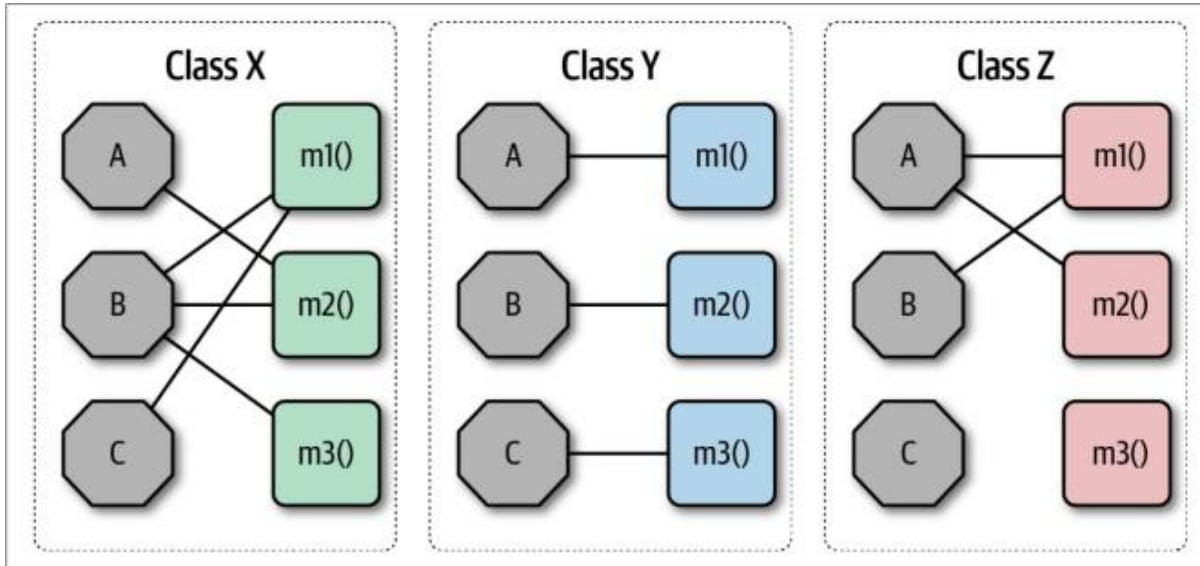
The example on the left shows a class consisting of methods A through E and variables x and y. A calls B and B accesses x. Both C and D access y. D calls E, but E doesn't access any variables. This class consists of 2 unrelated components (LCOM4=2) → You could split it as {A, B, x} and {C, D, E, y}.

In the example on the right, we made C access x to increase cohesion. Now the class consists of a single component (LCOM4=1). It is a cohesive class.

## 3.4. Measuring Modularity



### *The Lack of Cohesion in Methods - examples*



fields appear as single letters and methods appear as blocks.

Figure 3-1. Illustration of the LCOM metric, where fields are octagons and methods are squares

In Class X, the LCOM score is low, indicating good structural cohesion.

Class Y, however, lacks cohesion; each of the field/method pairs in Class Y could appear in its own class without affecting behavior.

Class Z shows mixed cohesion, where developers could refactor the last field/method combination into its own class.

# 3.4. Measuring Modularity



## *Cohesion Tools*

- ▶ JArchitect
- ▶ [TAACO](#)
- ▶ CK <https://github.com/mauricioaniche/ck>
- ▶ Metric Reloaded (IntelliJ IDEA plugin)
- ▶ Java Methods Reference Diagram (IntelliJ IDEA plugin)
- ▶ ...

# 3.4. Measuring Modularity



## Coupling

- ▶ Coupling is the degree of interdependence between software modules.
- ▶ Coupling metrics: afferent coupling and efferent coupling.
  - Afferent coupling measures the number of incoming connections to a code artifact (component, class, function, and so on).
  - Efferent coupling measures the outgoing connections to other code artifacts.
- ▶ For virtually every platform tools exist that allow architects to analyze the coupling characteristics of code in order to assist in restructuring, migrating, or understanding a code base.

### Why Such Similar Names for Coupling Metrics?

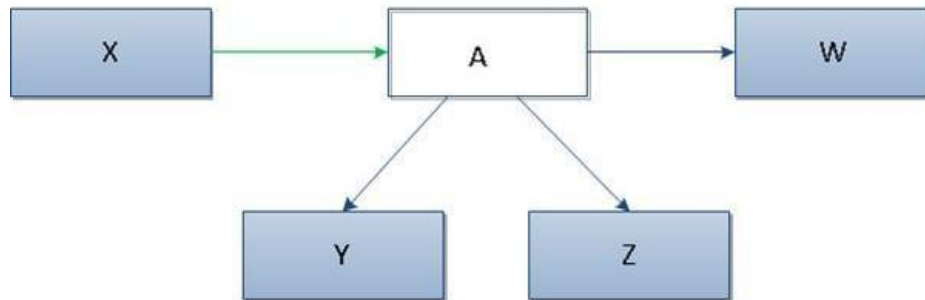
Why are two critical metrics in the architecture world that represent opposite concepts named virtually the same thing, differing in only the vowels that sound the most alike? These terms originate from Yourdon and Constantine's *Structured Design*. Borrowing concepts from mathematics, they coined the now-common afferent and efferent coupling terms, which should have been called incoming and outgoing coupling. However, because the original authors leaned toward mathematical symmetry rather than clarity, developers came up with several mnemonics to help out: *a* appears before *e* in the English alphabet, corresponding to *incoming* being before *outgoing*, or the observation that the letter *e* in efferent matches the initial letter in *exit*, corresponding to outgoing connections.

# 3.4. Measuring Modularity



## *Coupling – efferent ( $C^e$ ) & afferent ( $C^a$ )*

- ▶ Afferent coupling ( $C^a$ ): incoming dependency
- ▶ Efferent coupling ( $C^e$ ): outgoing dependency
- ▶ When a  $C^e > 20$  indicates instability of a component, changes in any external component may cause the need for changes within this component. The  $C^e$  value should be between 0 and 20, and higher values cause problems with development and maintenance.
- ▶ The higher the value of  $C^a$  means the stability of the component will be higher. This shows that there are many components depending on this component. The recommended value of  $C^a$  is around 0 to 500.
- ▶ Example:



$$C^e = 3$$
$$C^a = 1$$

# 3.4. Measuring Modularity

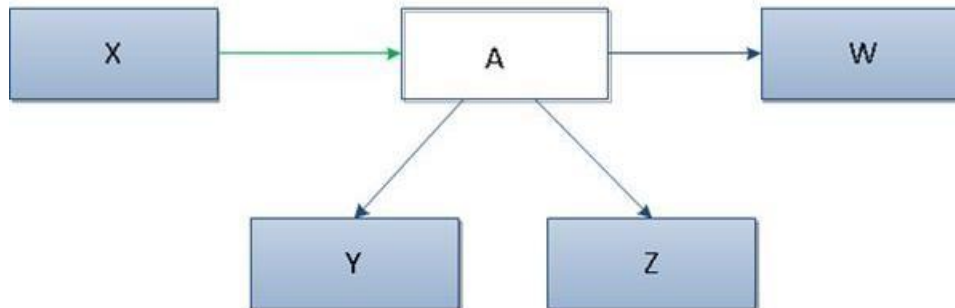


## Coupling - Instability

- ▶ Instability is defined as the ratio of efferent coupling to the sum of both efferent and afferent coupling.

$$I = \frac{C^e}{C^e + C^a}$$

- $C^e$  represents *efferent* (or outgoing) coupling, and  $C^a$  represents *afferent* (or incoming) coupling.
- ▶ The instability metric determines the volatility of a code base. A code base that exhibits high degrees of instability breaks more easily when changed because of high coupling.
  - For example, if a class calls to many other classes to delegate work, the calling class shows high susceptibility to breakage if one or more of the called methods change.



$$I = \frac{C^e}{C^e + C^a} = \frac{3}{3+1} = 0.75$$

## 3.4. Measuring Modularity



### *Coupling – Instability (cont.)*

- ▶ Components have many outgoing dependencies and not many incoming dependencies (I **approaching 1**), which are **unstable** because these components are highly susceptible to change;
- ▶ Components have many incoming dependencies and not many outgoing dependencies (I value is **close to 0**), so they will be **less likely to be changed**. However, changes from them will have a considerable impact.
- ▶ Preferred values for the instability (I) should be in the range from 0 to 0.3 or 0.7 to 1 (Components should be very stable or unstable). The **medium-stability components should be avoided**.

## 3.4. Measuring Modularity



### *Coupling - Abstractness*

- ▶ Abstractness is the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

$$A = \frac{N_a}{N_c}$$

- $N_a$ : the number of abstract classes and interfaces in the component,
  - $N_c$ : the total number of classes (abstract classes, concrete classes, and interfaces) in the component.
- *Example: consider a package with:*
    - 5 abstract classes and interfaces (abstract types)
    - 10 classes and interfaces (total types)
    - The abstractness would be:  $A = 5 / 10 = 0.5$

# 3.4. Measuring Modularity



## *Coupling – Abstractness (cont.)*

- ▶ The value of  $A$  ranges from 0 to 1.
  - A module with a high abstractness value (close to 1) indicates that it contains a significant number of abstract classes and interfaces, suggesting a **more abstract** and **less concrete** implementation.
  - Conversely, a low abstractness value (close to 0) indicates a **more concrete** implementation with **fewer abstract** classes and interfaces.
- ▶ The **goal of abstractness** is to strike a **balance** based on the design principles.
  - **High** abstractness can lead to more **flexibility** and **extensibility**, allowing for easier modifications and extensions.
  - However, **excessive** abstractness without concrete implementations may lead to a **lack of functionality**.
  - The **optimal level** of abstractness **depends** on the specific design goals and requirements of the software system.

# 3.4. Measuring Modularity

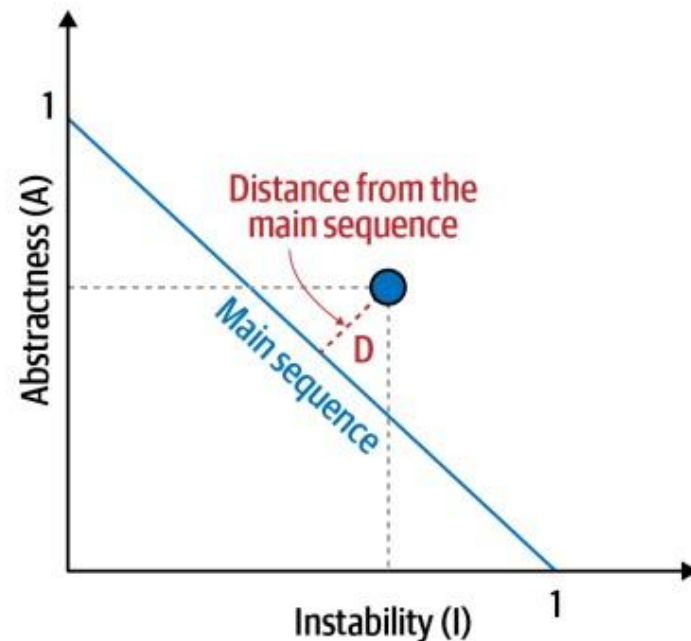
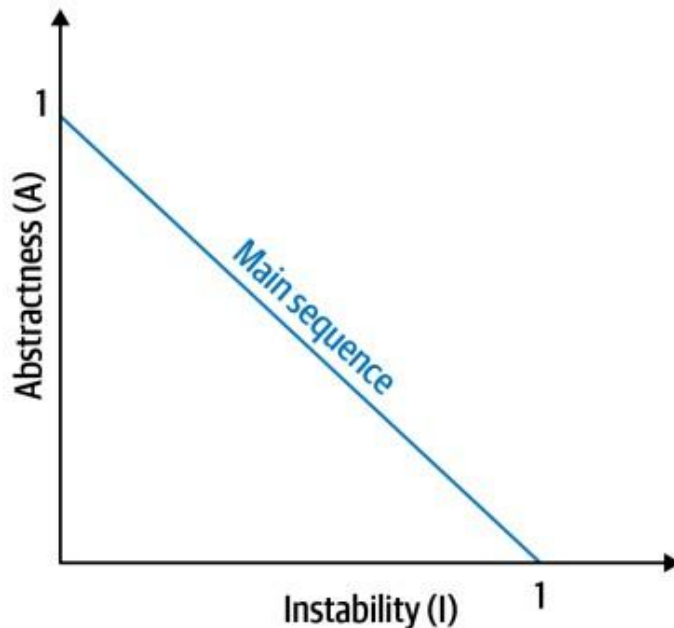


## *Coupling - Distance from the Main Sequence*

- ▶ One of a few holistic metrics architects have for architectural structure is the distance from the main sequence, a derived metric based on instability and abstractness.

$$D = |A + I - 1|$$

- $A$  = abstractness and  $I$  = instability.

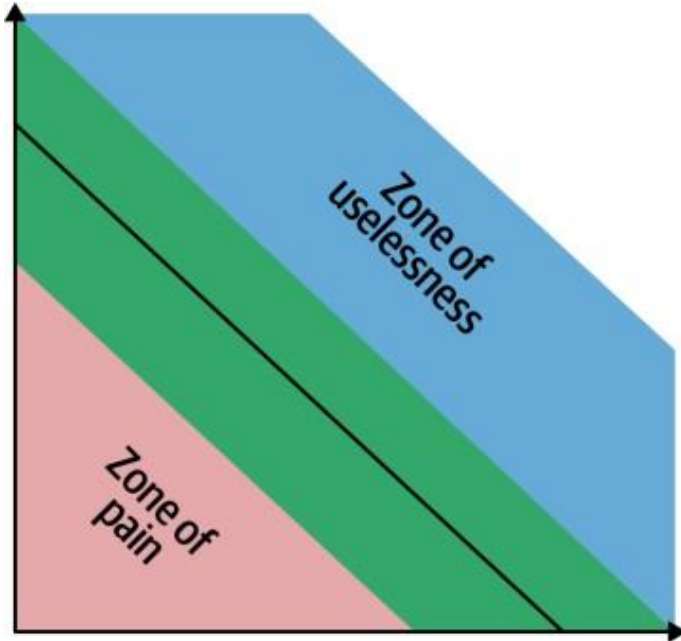


# 3.4. Measuring Modularity

## *Coupling - Distance from the Main Sequence*



- ▶ The closer to the line, the better balanced the class.
  - Classes that fall too far into the upper-righthand corner enter into what architects call the zone of uselessness: code that is too abstract becomes difficult to use.
  - Conversely, code that falls into the lower-lefthand corner enter the zone of pain: code with too much implementation and not enough abstraction becomes brittle and hard to maintain.



*Figure 3-4. Zones of Uselessness and Pain*

## 3.4. Measuring Modularity



### *Connascence*

- ▶ “Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system.” - Meilir Page-Jones
- ▶ Types of connascence
  - **Static:** source-code-level coupling. It is a refinement of the afferent and efferent couplings defined by Structured Design. In other words, architects view the following types of static connascence as the degree to which something is coupled, either afferently or efferently
  - **Dynamic:** execution-time coupling. It analyzes calls at runtime

# 3.4. Measuring Modularity



## *Static Connascence*

- ▶ Connascence of Name (CoN)
  - **Multiple components must agree on the name of an entity.**
  - Names of methods represents the most common way that code bases are coupled and the most desirable, especially in light of modern refactoring tools that make system-wide name changes trivial.
- ▶ Connascence of Type (CoT)
  - **Multiple components must agree on the type of an entity.**
  - This type of connascence refers to the common facility in many statically typed languages to limit variables and parameters to specific types. However, this capability isn't purely a language feature—some dynamically typed languages offer selective typing, notably Clojure and Clojure Spec.
- ▶ Connascence of Meaning (CoM) or Connascence of Convention (CoC)
  - **Multiple components must agree on the meaning of particular values.**
  - The most common obvious case for this type of connascence in code bases is hard-coded numbers rather than constants. For example, it is common in some languages to consider defining somewhere `int TRUE = 1; int FALSE = 0`. Imagine the problems if someone flips those values.

# 3.4. Measuring Modularity



## *Static Connascence*

- ▶ Connascence of Position (CoP)
  - **Multiple components must agree on the order of values.**
  - This is an issue with parameter values for method and function calls even in languages that feature static typing. For example, if a developer creates a method `void updateSeat(String name, String seatLocation)` and calls it with the values `updateSeat("14D", "Ford, N")`, the semantics aren't correct even if the types are.
- ▶ Connascence of Algorithm (CoA)
  - **Multiple components must agree on a particular algorithm.**
  - A common case for this type of connascence occurs when a developer defines a security hashing algorithm that must run on both the server and client and produce identical results to authenticate the user. Obviously, this represents a high form of coupling—if either algorithm changes any details, the handshake will no longer work.

# 3.4. Measuring Modularity



## *Dynamic Connascence*

- ▶ Connascence of Execution (CoE)
  - The **order of execution** of multiple components is **important**.
- ▶ Connascence of Timing (CoT)
  - The **timing of the execution** of multiple components is **important**.
  - The common case for this type of connascence is a race condition caused by two threads executing at the same time, affecting the outcome of the joint operation.
- ▶ Connascence of Values (CoV)
  - Occurs when **several values relate on one another** and **must change together**.
- ▶ Connascence of Identity (CoI)
  - Occurs when multiple components **must reference the same entity**.

# 3.4. Measuring Modularity



## *Connascence properties*

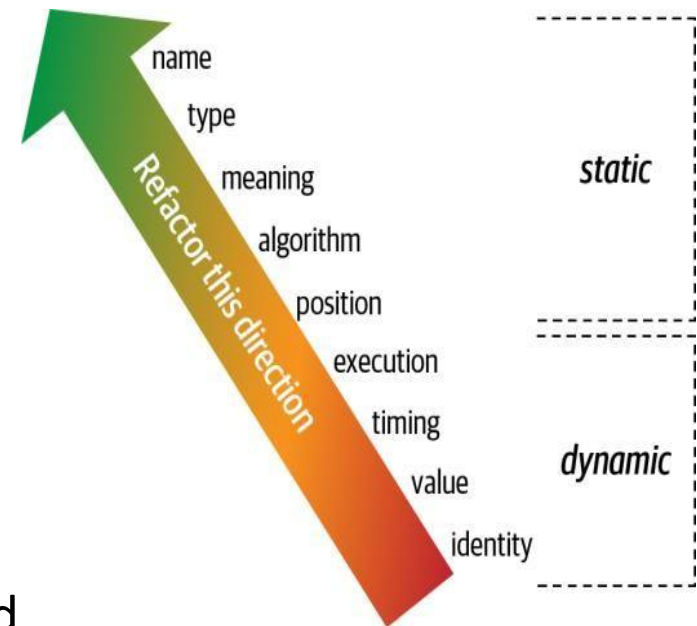
### ► Strength

- Strength of connascence is determined by the ease with which a developer can refactor that type of coupling
- Architects should prefer static connascence to dynamic connascence

### ► Locality

- The locality of connascence measures how proximal the modules are to each other in the code base.

Strength and locality should be considered together. Stronger forms of connascence found within the same module represent less code smell than the same connascence spread apart.



## 3.4. Measuring Modularity



### *Connascence properties*

#### ► Degree

The degree of connascence relates to the size of its impact—does it impact a few classes or many?

**Three guidelines** for using connascence to improve systems modularity:

1. Minimize overall connascence by breaking the system into encapsulated elements
2. Minimize any remaining connascence that crosses encapsulation boundaries
3. Maximize the connascence within encapsulation boundaries

Jim Weirich offers two great **pieces of advice**:

**Rule of Degree:** convert strong forms of connascence into weaker forms of connascence

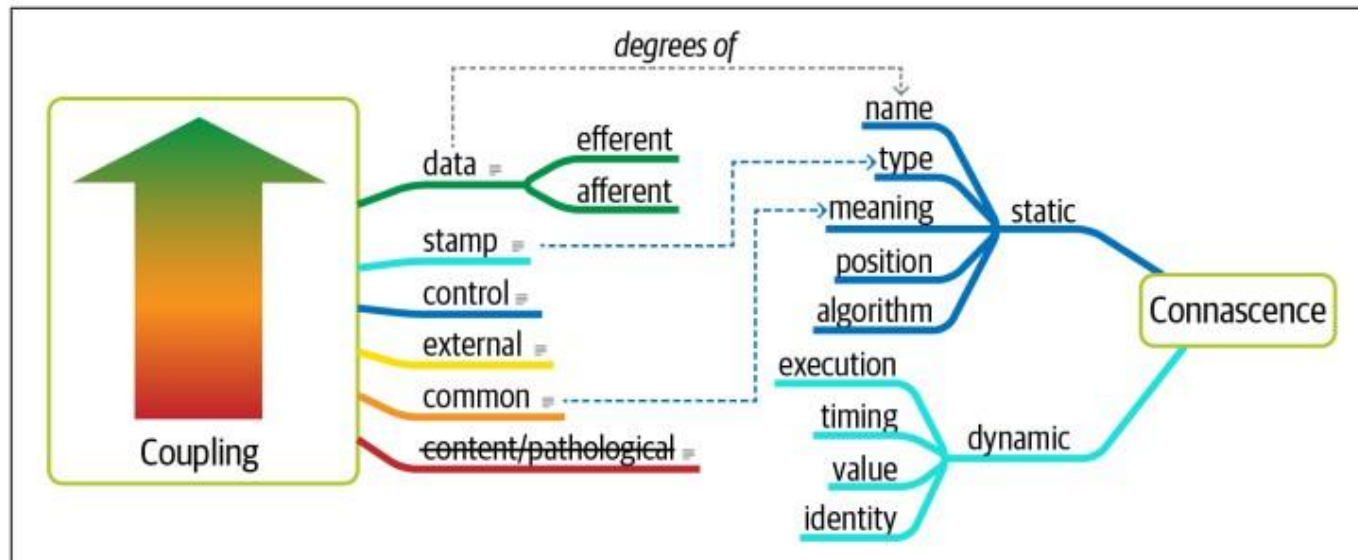
**Rule of Locality:** as the distance between software elements increases, use weaker forms of connascence

# Coupling & Connascence

## *Unifying Coupling and Connascence Metrics*



- ▶ These two views overlap.
  - What Page-Jones identifies as static connascence represents degrees of either incoming or outgoing coupling. **Structured programming only cares about in or out**, whereas **connascence cares about how things are coupled** together.





# Q&A